

ใบความรู้ที่ 9 การประมวลผลภาษาธรรมชาติ (Natural Language Processing: NLP)

วัตถุประสงค์การเรียนรู้

- เพื่อให้รู้แนวคิดและประโยชน์ของการนำโมเดลที่ฝึกสอนมาแล้วไปใช้งานต่อ (Transfer Learning)
- เพื่อให้เข้าใจกระบวนการทำงานของเทคนิค RAG (Retrieval-Augmented Generation) ในการเพิ่มบริบทความแม่นยำให้ AI
- เพื่อให้เข้าใจกลไกและขั้นตอนการปรับแต่ง (Fine-Tuning) โมเดลภาษาขนาดใหญ่ (LLM) สำหรับการใช้งานเฉพาะด้าน

1. ปรัชญาและรากฐานของการเรียนรู้แบบถ่ายโยงความรู้ (Transfer Learning)

การพัฒนาปัญญาประดิษฐ์ในยุคปัจจุบันได้ก้าวข้ามข้อจำกัดของการเริ่มต้นจากศูนย์ (Training from Scratch) กระบวนการสร้างโมเดลภาษาขนาดใหญ่ (Large Language Models: LLMs) จำเป็นต้องใช้ข้อมูลระดับล้านล้านโทเคนและทรัพยากรประมวลผลมหาศาล ซึ่งเป็นอุปสรรคสำหรับองค์กรทั่วไป แนวคิดการเรียนรู้แบบถ่ายโยงความรู้ (Transfer Learning) จึงถือกำเนิดขึ้น เพื่อปฏิวัติกระบวนการนี้

1.1. โมเดลพื้นฐาน (Foundation Models) และการปรับจูน (Fine-Tuning)

Transfer Learning อาศัยหลักการนำ "โมเดลที่ผ่านการฝึกฝนมาแล้ว" (Pre-trained Model หรือ Foundation Model) ซึ่งมีความรู้ทางไวยากรณ์และตรรกะพื้นฐานในระดับสูง มาใช้เป็นรากฐาน จากนั้นจึงนำมาผ่านกระบวนการ "ปรับแต่งเฉพาะทาง" (Fine-Tuning) ด้วยชุดข้อมูลที่มีขนาดเล็กกว่าและมีความเฉพาะเจาะจงกับบริบทขององค์กร วิธีนี้ช่วยประหยัดเวลา ทรัพยากร และให้ผลลัพธ์ที่มีความแม่นยำสูงในโดเมนเฉพาะ

1.2. หลักวิสัยธรรมในการพิจารณาเลือกโมเดล (Model Selection Criteria)

การนำโมเดลแบบโอเพนซอร์สมาใช้งาน จำเป็นต้องพิจารณาตัวแปรสำคัญ 3 ประการ ได้แก่ :

1.2.1. ขนาดของพารามิเตอร์ (Parameter Size): มักระบุด้วยตัวอักษร B (Billion) เช่น 7B, 13B หรือ 70B พารามิเตอร์คือจุดเชื่อมต่อในโครงข่ายประสาท ยิ่งมีจำนวนมาก โมเดลยิ่งมีความสามารถในการทำความเข้าใจบริบทที่ซับซ้อน แต่ก็ต้องการทรัพยากรคอมพิวเตอร์ที่สูงขึ้นเป็นเงาตามตัว

1.2.2. ข้อจำกัดด้านฮาร์ดแวร์ (Hardware Constraints): การรันโมเดลอาศัยหน่วยความจำวิดีโอ (VRAM) ของหน่วยประมวลผลกราฟิก (GPU) เป็นหลัก หากทรัพยากรฮาร์ดแวร์มีจำกัด วิศวกรจำเป็นต้องพิจารณาใช้เทคนิคการบีบอัดโมเดล (Quantization) หรือสลับไปใช้บริการผ่าน API ของผู้ให้บริการระบบคลาวด์แทน

1.2.3. ประเภทของโมเดล (Base vs. Instruct):

1.2.3.1. Base Model: เป็นโมเดลดั้งเดิมที่ถูกฝึกมาเพื่อ "ทำนายคำถัดไป" (Next-token prediction) มักไม่มีความสามารถในการโต้ตอบหรือปฏิบัติตามคำสั่ง เหมาะสำหรับนำไปเป็นโครงสร้างพื้นฐานในการทำ Fine-Tuning ต่อ

1.2.3.2. Instruct Model: เป็นโมเดลที่ผ่านการปรับจูนมาแล้วให้สามารถเข้าใจเจตนาและปฏิบัติตามคำสั่ง (Instruction Following) ของมนุษย์ได้ เหมาะสำหรับการนำไปสร้างระบบแชทบอทโดยตรง

2. นวัตกรรมสถาปัตยกรรมของโมเดลภาษาขนาดใหญ่ยุคใหม่ (State-of-the-Art LLM Architectures)

โมเดลภาษายุคใหม่ เช่น สถาปัตยกรรมตระกูล Gemma ได้รับการออกแบบเชิงวิศวกรรมเพื่อแก้ไขปัญหาคอขวดของสถาปัตยกรรม Transformer ดั้งเดิม โดยเฉพาะในด้านการประมวลผลข้อมูลขนาดยาวและการรับรู้แบบพหุวิถีส (Multimodal)

2.1. สถาปัตยกรรมการรับรู้ภาพและข้อความ (Vision-Language Architecture)

โมเดลระดับก้าวหน้าได้บูรณาการตัวเข้ารหัสภาพ (SigLIP Vision Encoder) เข้ากับโมเดลภาษา การประมวลผลภาพจะอาศัยอัลกอริทึมในการครอบตัดและปรับขนาดภาพ (Pan&Scan) ให้ได้สัดส่วนมาตรฐาน จากนั้นภาพจะถูกบีบอัดให้กลายเป็น "โทเคนแบบอ่อน" (Soft tokens) จำนวนจำกัดผ่าน MultiModalProjector เพื่อส่งเข้าไปประมวลผลร่วมกับโทเคนภาษา ช่วยลดภาระการคำนวณในชั้นโครงข่าย

2.2. กลไกความสนใจแบบผสมผสาน (Interleaved Attention Mechanism)

ปัญหาใหญ่ของการประมวลผลเอกสารขนาดยาวคือหน่วยความจำ KV-cache ที่ใช้เก็บสถานะความสัมพันธ์ของคำจะพุ่งสูงขึ้นแบบทวีคูณ สถาปัตยกรรมใหม่จึงเปลี่ยนจากการใช้ความสนใจแบบครอบคลุม

(Global Attention) ล้วน ๆ มาเป็นการใช้กลไก Interleaved Attention กลไกนี้มักถูกจัดสรรในอัตราส่วน 5 ต่อ 1 โดยประกอบด้วย:

2.2.1. ชั้นความสนใจแบบท้องถิ่น (Local Attention Layers): จำนวน 5 ชั้น ทำหน้าที่ใช้หน้าต่างแบบเลื่อน (Sliding Window) กวาดสายตาอ่านโทเคนที่อยู่ติดกัน เพื่อประหยัด KV-cache

2.2.2. ชั้นความสนใจแบบครอบคลุม (Global Attention Layer): จำนวน 1 ชั้น ทำหน้าที่รวบรวมบริบทจากทั่วทั้งเอกสารมาเชื่อมโยงกัน

2.3. การขยายหน้าต่างบริบท (Extended Context Window)

การผสมผสานกลไก Interleaved Attention ทำให้โมเดลสามารถขยาย Context Window ได้สูงสุดถึง 128,000 โทเคน ซึ่งเทียบเท่ากับการอ่านหนังสือนวนิยายทั้งเล่มในคราวเดียว กุญแจสำคัญทางคณิตศาสตร์คือการใช้เทคนิค RoPE Rescaling (Rotary Positional Embedding) โดยปรับขยายสเกลความถี่ฐาน (Base Frequency) ในชั้น Global Attention จาก 10,000 ขึ้นเป็น 1,000,000 เพื่อรักษาเสถียรภาพในการระบุตำแหน่งคำที่อยู่ห่างไกลกัน

3. สถาปัตยกรรมการดึงข้อมูลเพื่อเสริมการสร้างคำตอบ (Retrieval-Augmented Generation: RAG)

ข้อบกพร่องที่ร้ายแรงที่สุดของ LLMs คือปรากฏการณ์ การหลอนข้อมูล (AI Hallucination) ซึ่งเกิดจากการที่โมเดลพยายามคาดเดาคำตอบในเรื่องที่ตนเองไม่เคยเรียนรู้มาก่อน ทำให้ได้ข้อมูลที่ผิดเพี้ยนจากความเป็นจริง เพื่อแก้ปัญหานี้โดยไม่ต้องเสียทรัพยากรไปกับการทำ Fine-Tuning ใหม่ทั้งระบบ สถาปัตยกรรม RAG จึงถูกนำมาประยุกต์ใช้

หลักการของ RAG คือการแยกหน่วยความจำออกจากหน่วยประมวลผล เมื่อมีคำถามเข้าสู่ระบบ อัลกอริทึมจะไปค้นหาข้อความที่เกี่ยวข้องที่สุดจากคลังเอกสารภายนอก (Retrieval) และนำข้อความนั้นมาเสริมเป็นบริบท (Augment) บังคับให้ AI สร้างคำตอบ (Generation) โดยอ้างอิงจากฐานข้อมูลที่ป้อนให้เท่านั้น

3.1. วิสวกรรมจัดการข้อความ (Text Chunking and Overlap Strategies)

การป้อนเอกสารความยาวหลายร้อยหน้าเข้าสู่ AI ย่อมเกินขีดจำกัดของ Context Window ดังนั้นข้อมูลจึงต้องผ่านกระบวนการ "แบ่งส่วนข้อมูล" (Chunking)

3.1.1. Chunk Size (ขนาดส่วนย่อย): คือการกำหนดขอบเขตความยาวสูงสุดของข้อความแต่ละท่อน (เช่น 800 ตัวอักษร) เพื่อให้โมเดลสามารถจับใจความได้ครบถ้วนโดยไม่สูญเสียความหมาย

3.1.2. Chunk Overlap (ระยะเหลื่อมซ้อน): หากข้อความถูกตัดขาดแบบตรงตัว อาจทำให้ประโยคสำคัญที่อยู่รอยต่อถูกผ่าครึ่ง การกำหนดให้จุดเริ่มต้นของท่อนถัดไป "ถอยหลัง" กลับมาทับซ้อนกับท่อนก่อนหน้า (เช่น 200 ตัวอักษร) จะช่วยรักษาความต่อเนื่องของบริบท (Context Continuity) ป้องกันไม่ให้สาระสำคัญสูญหายไประหว่างกระบวนการค้นหา

3.2. คณิตศาสตร์ของการแปลงเวกเตอร์และการวัดความคล้ายคลึง

ข้อมูลข้อความที่ถูกแบ่งส่วนจะต้องผ่านกระบวนการ Embedding เพื่อเข้ารหัสเป็นชุดตัวเลขเวกเตอร์ (Vector Representation) และต้องผ่านการปรับบรรทัดฐาน (Normalization) ให้เวกเตอร์มีความยาวเท่ากับ 1 หน่วย เพื่อเพิ่มประสิทธิภาพในการคำนวณ

การค้นหาข้อมูลในระบบ RAG อาศัยคณิตศาสตร์เชิงพื้นที่ที่เรียกว่า Cosine Similarity เพื่อเปรียบเทียบหาความคล้ายคลึงระหว่างเวกเตอร์ของคำถาม (Query) และเวกเตอร์ของเนื้อหา (Passage)

สูตรของ Cosine Similarity คือการหาผลคูณเชิงสเกลาร์ (Dot Product) ระหว่างเวกเตอร์สองเส้นที่ถูกปรับบรรทัดฐานแล้ว

$$\cos(\theta) = \frac{A \cdot B}{||A|| ||B||}$$

ค่าเข้าใกล้ 1: เวกเตอร์ทำมุม 0 องศา ชี้ไปในทิศทางเดียวกัน หมายถึงข้อความมีความสอดคล้องกันทางความหมายมากที่สุด

ค่าเข้าใกล้ 0: เวกเตอร์ทำมุมตั้งฉากกัน หมายถึงข้อความไม่มีความเกี่ยวข้องกันเลย

ค่าเป็น -1: เวกเตอร์ทำมุม 180 องศา หมายถึงข้อความมีความหมายไปในทิศทางตรงกันข้าม

ระบบจะจัดเรียงคะแนนจากมากไปน้อย และดึงเอาข้อความที่มีคะแนนสูงสุด (Top-K) มาใช้เป็นบริบทในการสร้างคำตอบต่อไป

3.3. วิศวกรรมชุดคำสั่งระหว่างการอนุมาน (Prompt Engineering and Inference)

ในขั้นตอนการอนุมาน (Inference) หรือการให้ AI สร้างคำตอบ การตีกรอบบริบทถือเป็นขั้นตอนสำคัญสูงสุด วิศวกรจะต้องประกอบรวมชุดคำสั่ง (Prompt) อย่างรัดกุม โดยระบุเงื่อนไขอย่างชัดเจน เช่น "ให้ตอบคำถามโดยอ้างอิงจากข้อมูลที่ให้ไว้เท่านั้น" พร้อมแนบเนื้อหาที่ค้นหามาได้

นอกจากนี้ ในเชิงวิศวกรรมระบบ จำเป็นต้องมีการบริหารจัดการทรัพยากร เช่น การส่งสัญญาณปิดการคำนวณความชัน (torch.no_grad()) เพื่อแจ้งให้เฟรมเวิร์กทราบว่ากระบวนการนี้ไม่ใช่การฝึกฝนโมเดล ซึ่งจะช่วยปลดล็อกหน่วยความจำที่สงวนไว้ และทำให้การสร้างคำตอบรวดเร็วขึ้น

4. ทฤษฎีการปรับแต่งพารามิเตอร์แบบมีประสิทธิภาพ (Parameter-Efficient Fine-Tuning: PEFT)

เมื่อวัตถุประสงค์คือการปรับเปลี่ยนรูปแบบการตอบสนอง พฤติกรรม หรือสร้างบุคลิกเฉพาะทางให้ AI การใช้ RAG ย่อมไม่ตอบโจทย์ จำเป็นต้องใช้วิธีการปรับจูนโมเดล (Fine-Tuning) แต่การปรับค่าน้ำหนักทุกพารามิเตอร์ในโมเดลขนาดใหญ่ (Full Fine-Tuning) ต้องใช้พื้นที่หน่วยความจำมหาศาลในการจัดเก็บสถานะตัวปรับสมการ (Optimizer States) แนวทางแก้ปัญหาคือการใช้กระบวนการ PEFT โดยเฉพาะเทคนิค Low-Rank Adaptation (LoRA)

4.1. โครงสร้างและสมการของ LoRA

LoRA อาศัยทฤษฎีการแยกตัวประกอบเมทริกซ์ (Matrix Factorization) โดยมีหลักการคือการ "แช่แข็ง" (Freeze) เมทริกซ์ค่าน้ำหนักดั้งเดิม (W_0) ของโมเดลไว้ทั้งหมด และเพิ่มเมทริกซ์ขนาดเล็กเข้าไปทำงานคู่ขนานกันเฉพาะในทิศทางของความรู้ใหม่

สมการการอัปเดตค่าน้ำหนักของ LoRA กำหนดไว้ดังนี้:

$$W_{new} = W_0 + \Delta W = W_0 + BA$$

เมทริกซ์ A และ B : เป็นเมทริกซ์ที่มีระดับมิติต่ำ (Low-rank) ทำให้มีจำนวนพารามิเตอร์น้อยกว่าเมทริกซ์ W_0 อย่างมาก (ลดลงได้กว่า 99%)

การตั้งค่าเริ่มต้น (Initialization): เมทริกซ์ A จะถูกสุ่มค่าเริ่มต้นแบบเกาส์เซียน (Gaussian/Kaiming) ในขณะที่เมทริกซ์ B จะถูกตั้งค่าให้เป็น 0 ทั้งหมด เพื่อให้มั่นใจว่าในรอบแรกของการเรียนรู้ ผลคูณของ BA จะมีค่าเป็น 0 ซึ่งจะไม่ทำให้สถิติปัญญาเดิมของโมเดลผิดเพี้ยนไป

4.2. ไฮเปอร์พารามิเตอร์ Rank (r) และ Alpha (α)

พลวัตของการเรียนรู้ด้วย LoRA ถูกควบคุมผ่านพารามิเตอร์ 2 ตัว:

4.2.1. Rank (r): หรือระดับมิติ เป็นตัวกำหนดความจุในการรองรับรูปแบบความรู้ใหม่ ยิ่งค่า r สูง โมเดลยังสามารถเรียนรู้ความสัมพันธ์ที่ซับซ้อนได้ลึกซึ้ง แต่ก็แลกมากับปริมาณการคำนวณที่เพิ่มขึ้นและความเสี่ยงในการลืมข้อมูลเดิม (Catastrophic Forgetting)

4.2.2. Alpha (α): เป็นตัวประกอบมาตราส่วน (Scaling Factor) ที่คอยควบคุมอิทธิพลของเมทริกซ์ BA ที่จะเข้าไปแทรกแซงเมทริกซ์หลักมากน้อยเพียงใด โดยทั่วไปมักตั้งค่า α ให้มีค่าเป็นสองเท่าของ r (เช่น $r = 8, \alpha = 16$) เพื่อกระตุ้นให้โมเดลเปิดรับข้อมูลชุดใหม่ได้อย่างมีประสิทธิภาพ

5. วิศวกรรมการบีบอัดข้อมูลแบบควอนไทเซชันและ QLoRA (Quantization and QLoRA)

เพื่อให้การฝึกสอนโมเดลระดับหลายพันล้านพารามิเตอร์สามารถดำเนินการได้บนฮาร์ดแวร์ทั่วไป วิศวกรจะใช้เทคนิค QLoRA ซึ่งเป็นการผสมผสาน LoRA เข้ากับการบีบอัดสมอง AI ลงให้เหลือเพียง 4-bit (Quantization)

5.1. โครงสร้างคณิตศาสตร์ 4-bit NormalFloat (NF4)

การบีบอัดข้อมูลให้เหลือพื้นที่เพียง 16 ระดับแบบดั้งเดิมมักก่อให้เกิดความคลาดเคลื่อนสูง สถาปัตยกรรม NF4 แก้ปัญหานี้โดยอาศัยข้อค้นพบทางสถิติว่า ค่าน้ำหนักของโครงข่ายประสาทเทียมมักมีการกระจายตัวแบบระฆังคว่ำ (Normal Distribution)

กระบวนการของ NF4 จะแบ่งข้อมูลออกเป็นบล็อกย่อย (Block-wise) หาค่าสูงสุดสัมบูรณ์ (Absmax) แล้วปรับช่วงสเกลให้เป็น $[-1, 1]$ จากนั้นจึงสร้างตารางระดับควอนไทล์ (Quantile) 16 ระดับตามแนวพื้นที่ใต้กราฟระฆังคว่ำ เพื่อให้ข้อมูลส่วนใหญ่ที่เกาะกลุ่มกันบริเวณค่า 0 ได้รับการจัดสรรรหัสตัวเลขอย่างละเอียดยิ่งขึ้น ช่วยลดข้อผิดพลาดในการบีบอัดลงอย่างมหาศาล

5.2. กลไก Double Quantization และ Compute Dtype

เพื่อให้เกิดการรีดทรัพยากรขั้นสูงสุด QLoRA ได้นำเสนอเทคนิค Double Quantization ซึ่งเป็นการนำค่าคงที่ที่ย่อย (Constants) ที่เกิดขึ้นจากการบีบอัดในรอบแรก มาทำกระบวนการบีบอัดซ้อนอีกหนึ่งชั้นเพื่อประหยัดหน่วยความจำ

ในทางปฏิบัติ แม้จะจัดเก็บข้อมูลในรูปแบบ 4-bit แต่เมื่อถึงรอบวงจรการคำนวณทางคณิตศาสตร์ (Forward/Backward Pass) ระบบจำเป็นต้องรักษาสภาพความแม่นยำไว้ จึงมีการกำหนด

Compute Dtype เพื่อสั่งให้คอมพิวเตอร์คลายการบีบอัดกลับมาเป็นความละเอียด 16-bit (เช่น Float16 หรือ Bfloat16) ช่วยลดขนาดของข้อมูลได้

5.3. วิศวกรรมข้อมูลเพื่อการเตรียมพร้อมสำหรับการปรับจูน (Data Engineering for Fine-Tuning)

ความสำเร็จของการทำ Fine-Tuning ยังขึ้นอยู่กับคุณภาพของข้อมูลนำเข้า ข้อความดิบจะต้องผ่านการจัดรูปแบบเชิงวิศวกรรมดังนี้ :

5.3.1. การรวมโครงสร้างและโทเคนสิ้นสุด (Prompt Formatting & EOS Token): คู่คำถาม-คำตอบจะต้องถูกร้อยเรียงกันเป็นข้อความเดียว และที่สำคัญที่สุดคือต้องมีการระบุ โทเคนจบประโยค (EOS Token) ปิดท้ายเสมอ เพื่อวางกรอบกลไกทางสถิติให้โมเดลรับรู้ว่าการสร้างคำตอบสิ้นสุดลงแล้ว และป้องกันปัญหาการพ่นข้อความลากยาว (Run-on Generation)

5.3.2. การควบคุมมิติสมการ (Truncation and Padding): เพื่อให้คอมพิวเตอร์สามารถส่งข้อมูลไปประมวลผลเป็นกลุ่มก้อน (Batch) ได้ ข้อมูลทุกชุดต้องมีมิติเท่ากัน ข้อความที่ยาวเกินกำหนดจะถูกตัดทิ้ง (Truncation) และข้อความที่สั้นเกินไปจะถูกส่งให้เติมช่องว่าง (Padding) ให้ยาวพอดีกับขนาดมาตรฐาน

5.3.3. การกำจัดข้อมูลมนุษย์ (Column Removal): โครงข่ายประสาทต้องการเพียงรหัสตัวเลขโทเคนในการคำนวณ คอลัมน์ที่เป็นตัวอักษรสตริง (String) ดั้งเดิมทั้งหมดจึงต้องถูกระบุให้ตัดทิ้งออกจากระบบ เพื่อประหยัดพื้นที่หน่วยความจำระหว่างการฝึกสอน

6. การประเมินผลและการพาสานค่าน้ำหนักเพื่อนำไปใช้งาน (Evaluation and Deployment)

ขั้นตอนสุดท้ายในวงจรการพัฒนาระบบ AI คือการตรวจสอบความสมบูรณ์และจัดเตรียมสถาปัตยกรรมให้อยู่ในสถานะที่พร้อมใช้งานจริง

6.1. มาตรวัดอัตโนมัติ vs. การประเมินด้วยมนุษย์ (Automated Metrics vs. Human Evaluation)

การวิเคราะห์ผลสัมฤทธิ์ของโมเดลแบบ Generative AI เป็นศาสตร์ที่มีความละเอียดอ่อน มาตรวัดทางคอมพิวเตอร์แบบคลาสสิกที่อาศัยการนับคำทับซ้อน (เช่น BLEU หรือ ROUGE) นำเสนอข้อจำกัดอย่างร้ายแรง เนื่องจากการสร้างประโยคของ LLM สามารถพลิกแพลงคำศัพท์ (Paraphrasing) หรือเรียงโครงสร้างประโยคใหม่ได้ หากประโยคมีความหมายถูกต้องสมบูรณ์ 100% ต่อการตีความของมนุษย์ แต่ใช้คำศัพท์ไม่ตรงกับเฉลยในฐานข้อมูล ระบบคอมพิวเตอร์จะกดคะแนนให้ต่ำลงทันที ซึ่งไม่สะท้อนความเป็นจริง

ด้วยเหตุนี้ การประเมินโมเดลระดับสูงจึงยึดถือ การประเมินด้วยมนุษย์ (Human Evaluation) เป็นมาตรฐานทองคำ (Gold Standard) โดยเน้นไปที่การประเมินเชิงบริบทและการตีกรอบความรู้ (Grounding)

ปัญหาคลาสสิกที่มนุษย์ต้องทำหน้าที่ตรวจสอบคือการหลอนข้อมูลเชิงความรู้พื้นฐาน หรือปรากฏการณ์ "ลืมข้อมูลเดิมไม่หมด" (Catastrophic Forgetting Limitation) แม้โมเดลจะผ่านการฝึกด้วย LoRA ให้เป็นผู้เชี่ยวชาญเฉพาะทาง (เปรียบเสมือนการนำเชอร์รี่ไปวางบนไอศกรีมโดยไม่เปลี่ยนรสชาติไอศกรีม) แต่โมเดลก็ยังคงครอบครองความรู้จากอินเทอร์เน็ตดั้งเดิมใน Base Model อยู่ หากผู้ทดสอบป้อนคำถามดักทางนอกเหนือบริบท โมเดลอาจยังคงตอบได้ฉะฉาน วิธีแก้ไขเชิงวิศวกรรมคือการแทรกชุดข้อมูลเชิงหลอก (Adversarial Examples) ในชุดข้อมูลฝึกสอน เพื่อป้อนตรรกะบังคับให้ระบบรู้จักการ "ปฏิเสธ" การให้ข้อมูลนอกกรอบอย่างมีมารยาท

6.2. กระบวนการทางคณิตศาสตร์ในการผสานค่าน้ำหนัก (Model Merging Architecture)

ผลผลิตจากกระบวนการฝึกฝนแบบ LoRA จะอยู่ในรูปแบบของเมทริกซ์ส่วนขยายขนาดเล็กที่เรียกว่า "อแดปเตอร์" (Adapter Weights) หากนำไปใช้งานในลักษณะของการสวมทับ (Wrapping) ระหว่างการอนุมาน จะทำให้เกิดความหน่วงเวลา (Latency) ในการคำนวณ ดังนั้นในขั้นตอนสุดท้ายก่อนการนำไปใช้งานจริง (Deployment) วิศวกรข้อมูลจะต้องดำเนินการ ผสานค่าน้ำหนัก (Merge and Unload)

กระบวนการนี้จะทำการบวกรวมเมทริกซ์ความรู้ใหม่ของ Adapter เข้ากับเมทริกซ์น้ำหนักดั้งเดิมของ Base Model อย่างถาวร เพื่อให้เกิดเป็นสมองเนื้อเดียวกันเพียงก้อนเดียว

ข้อควรระวังในเชิงสถาปัตยกรรมฮาร์ดแวร์ขณะผสานค่าน้ำหนักคือ การรวมเมทริกซ์ความละเอียดสูง (16-bit) สองชุดเข้าด้วยกันบนหน่วยความจำ VRAM อาจทำให้การ์ดจอประสบภาวะหน่วยความจำล้น (Out of Memory) แนวปฏิบัติมาตรฐานจึงต้องสั่งการให้ระบบย้ายกระแสข้อมูลการคำนวณทั้งหมดไปพึ่งพาทรัพยากรของหน่วยประมวลผลกลาง (CPU) แทน (device_map="cpu") แม้จะใช้เวลาในการรวมเมทริกซ์นานขึ้น แต่ให้เสถียรภาพที่สมบูรณ์แบบ ท้ายที่สุด ไฟล์โมเดลฉบับสมบูรณ์พร้อมกับพจนานุกรมแปลภาษา (Tokenizer) จะถูกบันทึกและส่งออกสู่ระบบเซิร์ฟเวอร์ต่อไป

อ้างอิง

ArXiv. (2024). Article 2410.21228v2. <https://arxiv.org/html/2410.21228v2>

Busycaesar. (n.d.). *Embeddings & cosine similarity*. Dev.to.

<https://dev.to/busycaesar/embeddings-cosine-similarity-4541>

Charan4u. (n.d.). *Unlocking the power of cosine similarity: The heart of text understanding*.

Medium. <https://medium.com/@charan4u/unlocking-the-power-of-cosine-similarity-the-heart-of-text-understanding-eed427df745a>

Elaidouni, M. (n.d.). *4-Bit quantization models QLoRA*. GitHub Pages.

<https://manalelaidouni.github.io/4Bit-Quantization-Models-QLoRa.html>

Emergent Mind. (n.d.). *4-bit NormalFloat (NF4) quantization*.

<https://www.emergentmind.com/topics/4-bit-normalfloat-nf4-quantization>

Emergent Mind. (n.d.). *NormalFloat 4 (NF4)*.

<https://www.emergentmind.com/topics/normalfloat-4-nf4>

Google Developers. (n.d.). *Gemma explained: What's new in Gemma 3*. Google Developers

Blog. <https://developers.googleblog.com/gemma-explained-whats-new-in-gemma-3/>

Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2021). *LoRA: Low-rank adaptation of large language models*. arXiv.

<https://arxiv.org/abs/2106.09685>

Hugging Face. (n.d.). *Making LLMs even more accessible with bitsandbytes, 4-bit*

quantization and QLoRA. <https://huggingface.co/blog/4bit-transformers-bitsandbytes>

Hugging Face. (n.d.). *nn.Linear4bit*. bitsandbytes documentation.

<https://huggingface.co/docs/bitsandbytes/reference/nn/linear4bit>

IBM. (n.d.). *Chunking strategies for RAG with LangChain and watsonx.ai*.

<https://www.ibm.com/think/tutorials/chunking-strategies-for-rag-with-langchain-watsonx-ai>

Label Studio. (n.d.). *Automated metrics vs. human evaluation: When each is the right*

choice. <https://labelstud.io/learningcenter/automated-metrics-vs-human-evaluation-when-each-is-the-right-choice/>

National Institutes of Health. (n.d.). *Article PMC12649634*. PubMed Central.

<https://pmc.ncbi.nlm.nih.gov/articles/PMC12649634/>

Neural-Hacker. (n.d.). *LoRA*. Hugging Face Blog. [https://huggingface.co/blog/Neural-](https://huggingface.co/blog/Neural-Hacker/lora)

[Hacker/lora](https://huggingface.co/blog/Neural-Hacker/lora)

Thinking Machines. (n.d.). *LoRA*. <https://thinkingmachines.ai/blog/lora/>